inmind .academy

DevOps – Session 1

OVERVIEW

- What is DevOps
- What is Docker
- What is a Container
- Docker vs Virtual Machine
- Docker Installation
- Main Docker Commands
- Debugging a Container
- Docker Registry
- How to write a Dockerfile
- Demo Project Overview

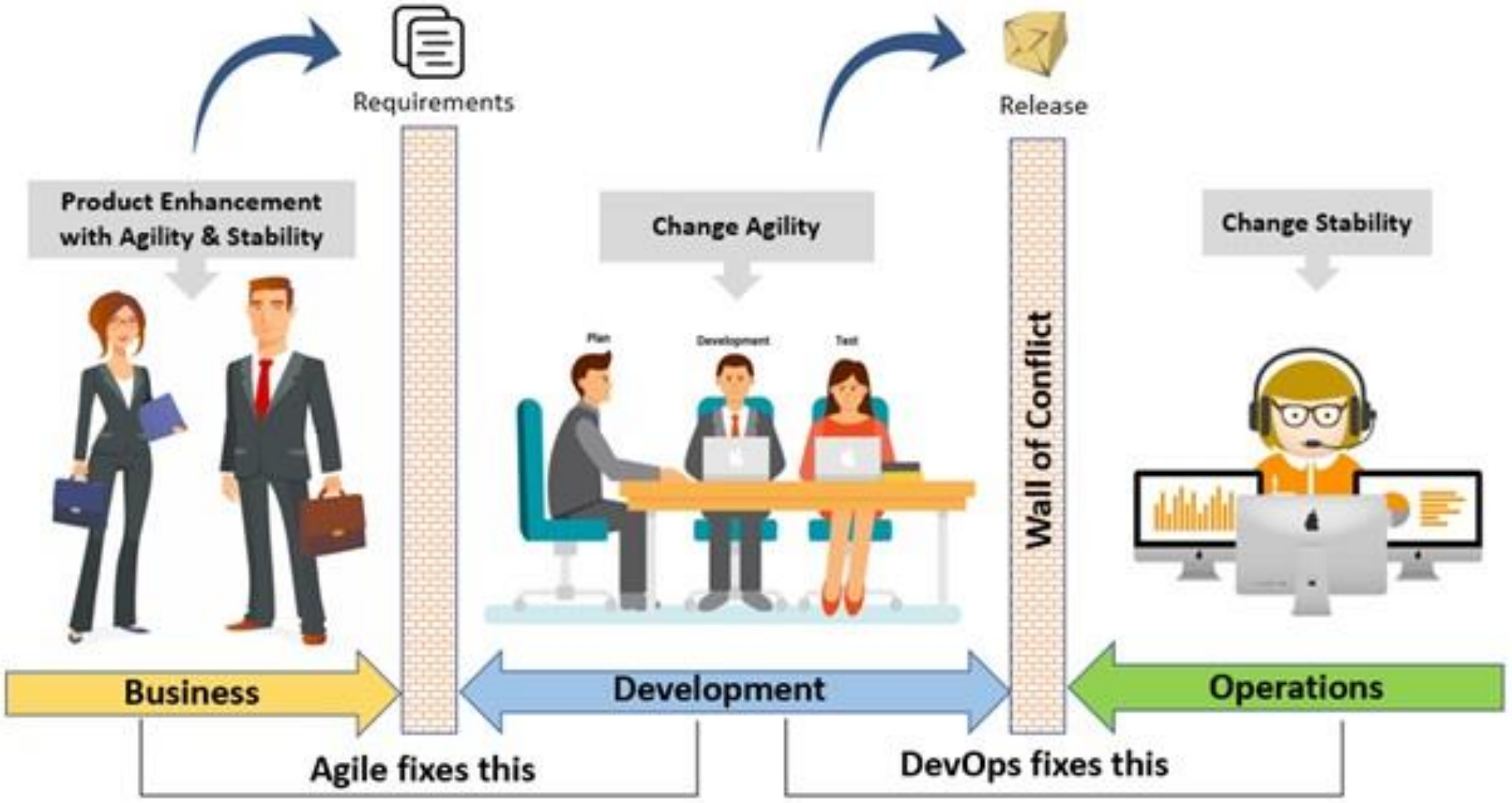
What is DevOps?

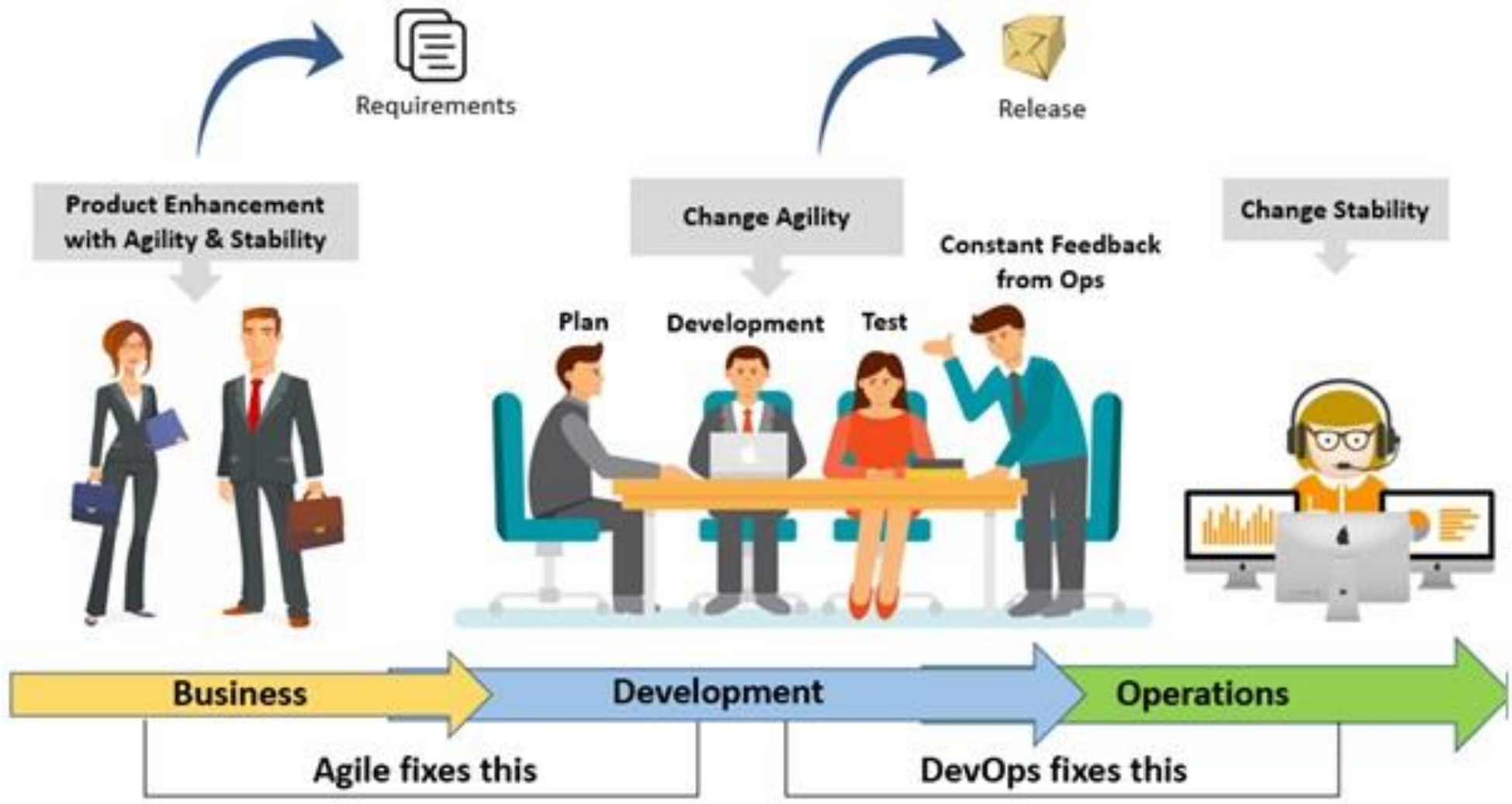
Dev - Ops

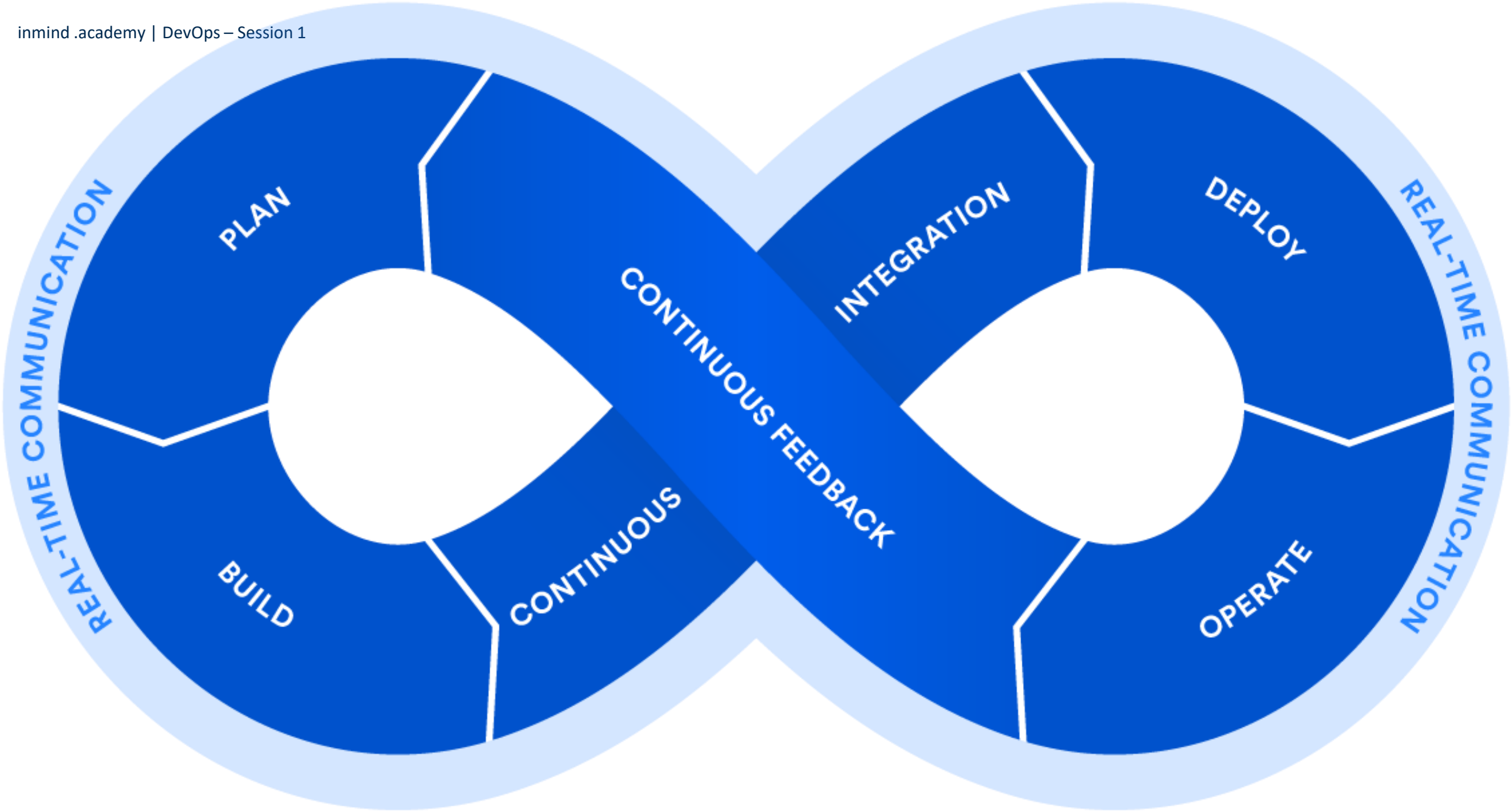

.NET
Core

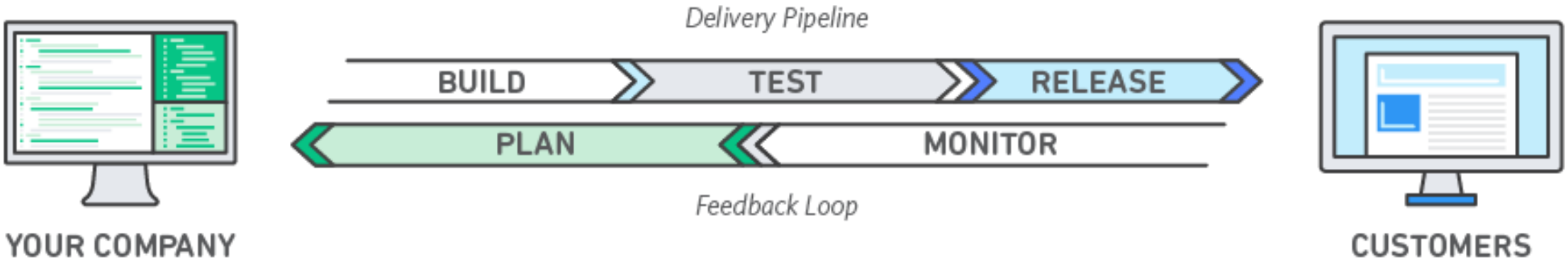

node
JS


php

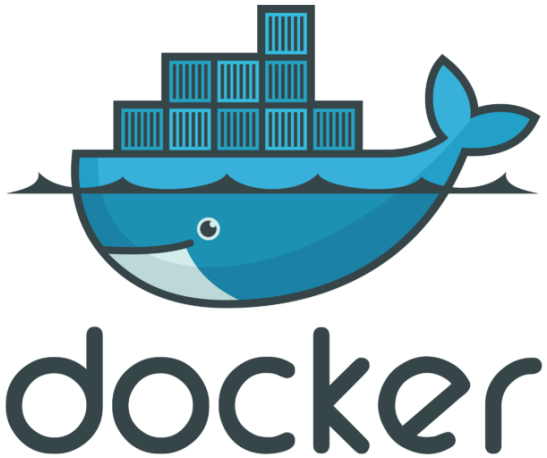

Requirements
Release
Product Enhancement with Agility & Stability
Change Agility
Change Stability
Wall of Conflict
Business
Development
Operations
Agile fixes this
DevOps fixes this


Requirements
Release
Product Enhancement with Agility & Stability
Change Agility
Change Stability
Constant Feedback from Ops
Plan
Development
Test
Business
Development
Operations
Agile fixes this
DevOps fixes this

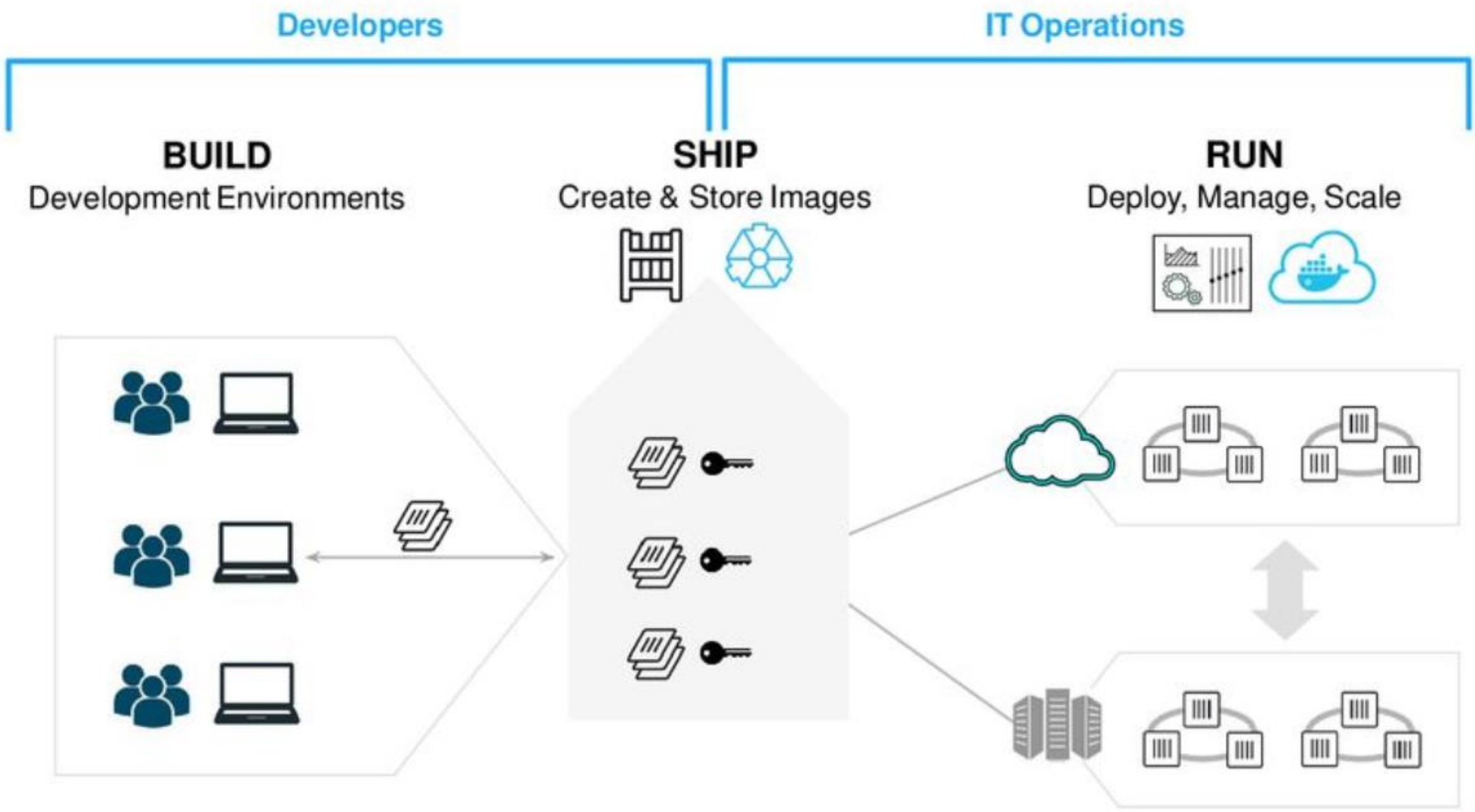

PLAN
CODE
BUILD
TEST
CONTINUOUS
INTEGRATION
DEPLOY
OPERATE
CONTINUOUS FEEDBACK
REAL-TIME COMMUNICATION
REAL-TIME COMMUNICATION


Delivery Pipeline
BUILD
TEST
RELEASE
PLAN
MONITOR
Feedback Loop
YOUR COMPANY
CUSTOMERS

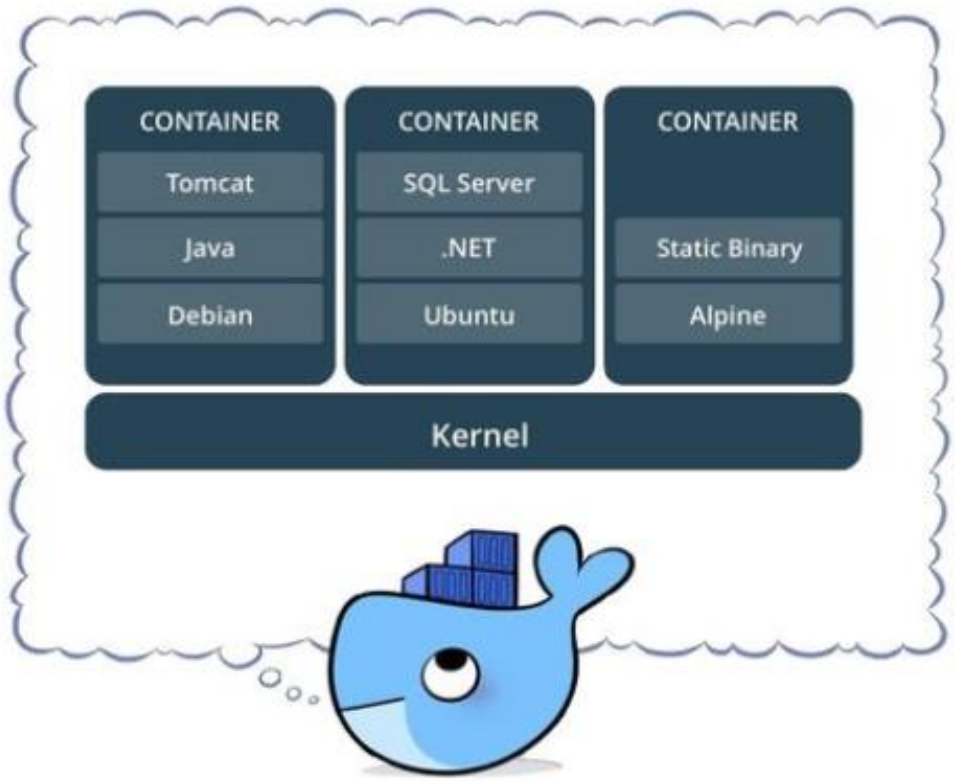
What is Docker



- Lightweight, open, secure platform
- Simply building, shipping, running apps
- Runs natively on Linux or Windows Server
- Runs on Windows or Mac Development machines (with a virtual machine)
- Relies on "images" and "containers"


Developers
IT Operations
BUILD
Development Environments
SHIP
Create & Store Images
RUN
Deploy, Manage, Scale

What is a Container



- Standardized packaging for software and dependencies
- Isolate apps from each other
- Share the same OS kernel
- Works for all major Linux distributions
- Containers native to Windows 2016

Docker vs VM

VMS	CONTAINERS
Heavyweight	Lightweight
Limited Performance	Native performance.
Each VM runs in its own OS	All containers share the host OS.
Hardware-level virtualization	OS virtualization.
Startup time in minutes	Startup time in milliseconds.
Allocates required memory	Requires less memory space.
Fully isolated and hence more secure	Process-level isolation, possibly less secure.

Docker Installation

Docker installation

```
sudo apt-get install docker-engine
```

```
sudo service docker start
```

```
sudo docker run hello-world
```

Main Commands

Docker Basics

docker image pull node:latest	docker container rm node
docker image ls	docker image rmi
docker run –d –p 8080:80 –name name	docker build –t node:2.0 .
docker container ps	docker image push node:2.0
docker container stop node (or id)	docker --help

Debugging a Container

Docker logs

To access the logs of a container run:

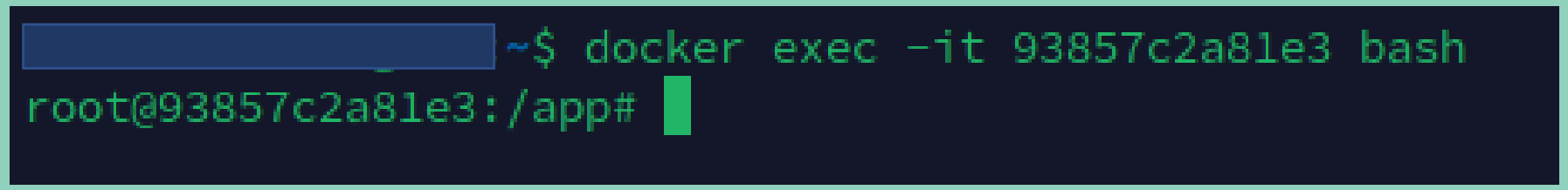
docker logs <container id> (or name)

```
[redacted]:~$ docker logs 93857c2a81e3
warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
      Storing keys in a directory '/root/.aspnet/DataProtection-Keys' that may not be persisted outside of the container. Protected data will be unavailable when container is destroyed.
info: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[62]
      User profile is available. Using '/root/.aspnet/DataProtection-Keys' as key repository; keys will not be encrypted at rest.
info: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[58]
      Creating key {c2453835-64e0-4421-9c5f-8ea2fcb1fbab} with creation date 2022-07-04 15:35:00Z, activation date 2022-07-04 15:35:00Z, and expiration date 2022-10-02 15:35:00Z.
warn: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[35]
      No XML encryptor configured. Key {c2453835-64e0-4421-9c5f-8ea2fcb1fbab} may be persisted to storage in unencrypted form.
info: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[39]
      Writing data to file '/root/.aspnet/DataProtection-Keys/key-c2453835-64e0-4421-9c5f-8ea2fcb1fbab.xml'.
info: Microsoft.EntityFrameworkCore.Infrastructure[10403]
      Entity Framework Core 5.0.15 initialized 'ImAcademyDbContext' using provider 'Npgsql.EntityFrameworkCore.PostgreSQL' with options: None
warn: Microsoft.AspNetCore.Server.Kestrel[0]
      Overriding address(es) 'http://+:80'. Binding to endpoints defined in UseKestrel() instead.
Hosting environment: Development
Content root path: /app
Now listening on: http://0.0.0.0:80
```

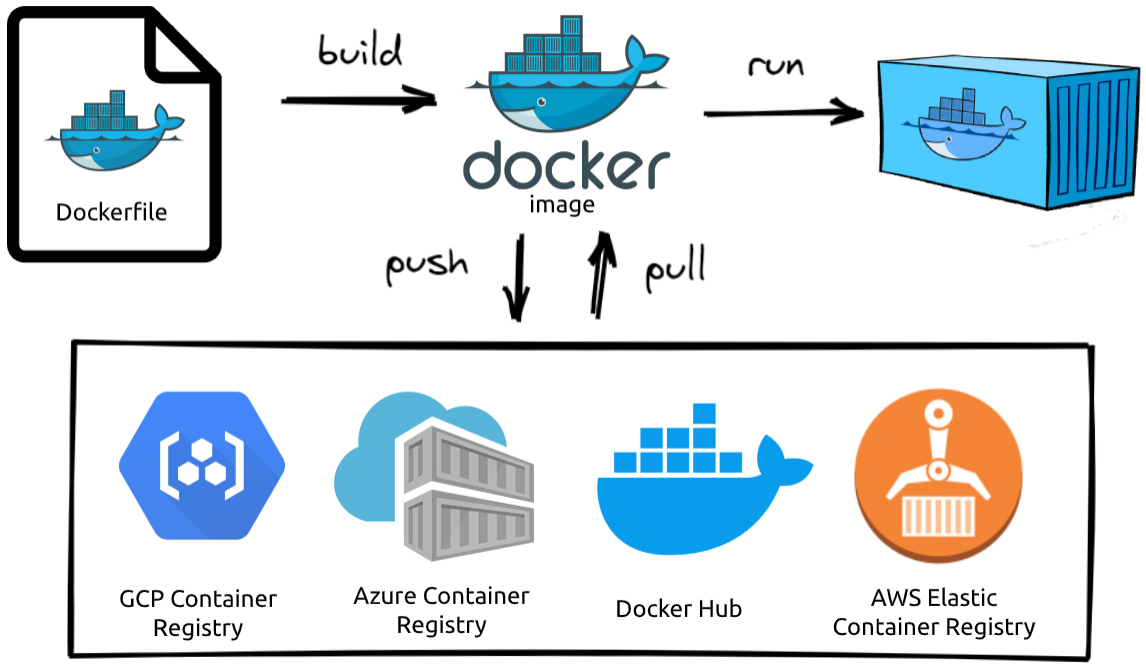
Docker exec

You can access the container itself by running:

```
docker exec –it <container id> bash (or sh)
```



Docker Registry



- Dockerhub is the official registry for all docker images
- A storage and distribution system for Docker
- Solution to integrate with and complement your CI/CD system
- Best way to distribute images inside an isolated network
- Multiple repos and projects with access management
- Public and private availability

Basic commands

docker login <registry>	docker logout <registry
docker build –t <tag> .	docker commit <container> <image>
docker tag <image> <new_tag>	docker cp <container>:<src> <dest>
docker push <registry>/<image>:<tag>	docker cp <src> <container>:<dest>
docker pull <registry>/<image>:<tag>	docker exec -it <container> <command>

How to write a Dockerfile

FROM

- Sets the base image for subsequent instructions
- Usage: FROM <image>
- Example: FROM ubuntu
- Needs to be the first instruction of every Dockerfile

```
# syntax=docker/dockerfile:1
FROM ubuntu:18.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

COPY

- Add files from your Docker client's current directory
- Usage: COPY <src code dir> <workdir>
- Example: COPY . /app

```
# syntax=docker/dockerfile:1
FROM ubuntu:18.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

RUN

- Executes any commands on the current image and commit the results
- Usage: RUN <command>
- Example: RUN apt-get install python3

```
# syntax=docker/dockerfile:1
FROM ubuntu:18.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

CMD

- Specifies what command to run inside the container
- Usage: CMD <command>
- Example: CMD python /app/app.py

```
# syntax=docker/dockerfile:1
FROM ubuntu:18.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

Additional

Label author

Specify the author of the file

Env <variable>

Specify environment variables

Expose <Number>

Specify the port that will be listening

Entrypoint [" ", " "]

Fire the executable file (ex: ["Node", "server.js"])

Multi-stage Dockerfile

- Specifies what command to run inside the container
- Usage: CMD <command>
- Example: CMD python /app/app.py

```
### STAGE 1:BUILD ###
# Defining a node image to be used as giving it an alias of "build"
# Which version of Node image to use depends on project dependencies
# This is needed to build and compile our code
# while generating the docker image
FROM node:12.14-alpine AS build
# Create a Virtual directory inside the docker image
WORKDIR /dist/src/app
# Copy files to virtual directory
# COPY package.json package-lock.json ./
# Run command in Virtual directory
RUN npm cache clean --force
# Copy files from local machine to virtual directory in docker image
COPY . .
RUN npm install
RUN npm run build --prod


### STAGE 2:RUN ###
# Defining nginx image to be used
FROM nginx:latest AS ngi
# Copying compiled code and nginx config to different folder
# NOTE: This path may change according to your project's output folder
COPY --from=build /dist/src/app/dist/my-docker-angular-app /usr/share/nginx/html
COPY /nginx.conf  /etc/nginx/conf.d/default.conf
# Exposing a port, here it means that inside the container
# the app will be using Port 80 while running
EXPOSE 80
```